

Where the Time Goes in Voice-to-Video

A technical report on CPU and GPU rendering

Rajesh Kumar Kona

<https://github.com/0KRK0/Voice-to-Video>

Abstract

This is the long version of how rendering works in Voice-to-Video, the narration-to-video system I built, and what I found while making it faster. The short version is the preprint *Where the Time Goes: Profiling, Segmenting and Verifying CPU and GPU Rendering in a Narrated-Video Pipeline*; this report keeps the details that did not fit there. The renderer composes every frame with Pillow and encodes with x264. Profiling showed composition taking 77% of render time on a real project, and 54–90% on a cloud container and 61–93% on my laptop depending on content, so a hardware encoder could not help much, and I spent the effort elsewhere: removing wasted buffer work (12 to 26 frames/s), splitting renders into checkpointed 12-second segments, and moving photograph resampling to an OpenGL 3.3 painter that must match the CPU within 2 levels per channel with no exceptions. On a GeForce GTX 1650 the first hardware run passed 15 of 23 test scenes; the pixel counts pointed to four bugs, and after fixing them all 23 pass, on the GTX 1650 and on Mesa llvmpipe. Measured on the same laptop at 1080p, the GPU composes photographs 11.7× faster than the CPU, transitions 7.7× and mixed content 3.0×, while typography stays on the CPU. That moves the bottleneck to the encoder for photograph-heavy videos. Two- and four-hour renders finished with flat memory and exact duration. Every number here comes from a file in the repository; Appendix A lists which.

1 What Voice-to-Video does, and why rendering matters

Voice-to-Video (VTV) takes a recording, a pasted script or a document and turns it into a narrated video. It transcribes the speech, groups it into scenes by idea, picks a visual for each scene (a drawn chart or diagram, a licensed photograph, typography or a generated image) and writes down why, checks that any number or date on screen comes from the source, lays everything on a timeline timed to the voice, and renders an MP4 with captions and credits.

Everything upstream of rendering costs roughly the same however long the video is. Rendering does not. A forty-minute video at 30 frames/s is 72,000 frames, and before the work in this report such a video took about four times its own length to render, on one core, while the other cores sat idle. That was the complaint users felt most.

VTV is a Python 3.11 codebase of 173 source files (63,528 lines) with 1,708 test functions. Two rules from early on shape the renderer: *the narration is the only clock*, and *the renderer draws every frame itself* instead of stitching encoded clips, so a transition is just a mix of two frames and there is no second timeline for the audio to drift against.

2 The starting point

The first renderer drew each frame with Pillow and piped raw pixels into one FFmpeg process that also added the narration:

```
ffmpeg -f rawvideo -pix_fmt rgb24 -s WxH -r fps -i - -i narration -c:v libx264
-preset veryfast -crf {18|23|30} -af "adelay={ms}:all=1,apad" -c:a aac -shortest out.mp4
```

It was simple and correct: apad before -shortest keeps deliberate visual time after the voice ends. It had two problems. A render was all or nothing, so a crash at 97% of a three-hour encode started again from frame zero. And it used one core. Threads would not have fixed that, because

composition is Pillow and Python, and the global interpreter lock serialises exactly the slow part. An audit in August 2026 also found renders running inside the API's event loop, so one 25–30 s render stalled every other request; I moved rendering into a separate worker process behind a durable SQLite queue.

3 Where the time goes

3.1 The project profile

Before picking an accelerator I profiled a real 1080p project. Composing frames took **77%** of render time and x264 took **23%**. Inside composition, drawing text was only 0.7%. The time went on moving 1920×1080 buffers around: a paste, a blend, a channel split and a colour conversion per frame. Most of that was redundant, and removing it took composition from 12 to 26 frames/s on the same machine, a $2.2 \times$ gain before touching any hardware.

Two smaller fixes came from the same method. Frames made of a single layer now skip an RGBA round trip, worth about 10 ms of a 42 ms frame at 1080p. The vignette mask, which cost 19 ms per frame (43 s over a 76-second video), is now cached and applied as one alpha composite on opaque frames. Tests check that both fast paths give the same pixels as the slow ones.

3.2 Per content kind

Composition cost depends on what is on screen, so I measured it on four synthetic workloads from the repository's benchmark: animated typography; a photograph under a slow zoom; a dissolve between two photographs; and alternating typography and photograph-with-caption frames. The photograph is twice the frame size. For each workload I composed 24 frames on one core and encoded the same frames with x264 (veryfast, CRF 23, one thread), taking the median of three repeats. I ran this on a 2-vCPU cloud container and again, as part of the GPU benchmark, on my laptop. Figure 1 shows the container at 1080p.

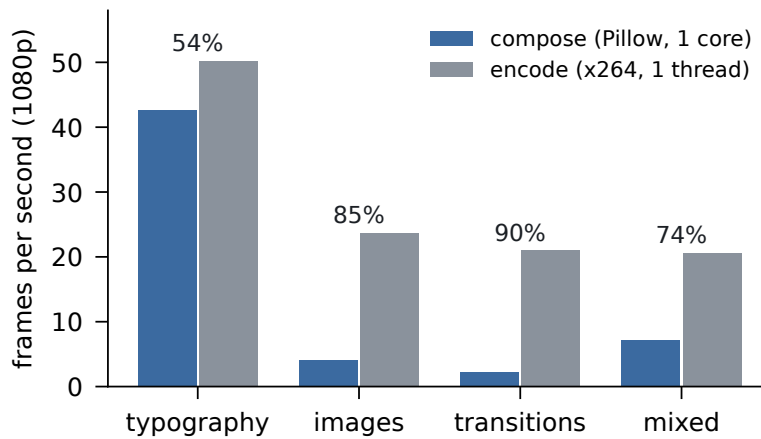


Figure 1: Composition (one core) and x264 encoding (one thread) at 1080p on the 2-vCPU container. Labels give composition's share of CPU time.

Composition's share at 1080p was 54% for typography, 85% for images, 90% for transitions and 74% for mixed content on the container, and 61%, 87%, 93% and 76% on the laptop. Mixed content sits close to the project's 77%.

3.3 Why not a hardware encoder

By Amdahl's law, if composition is 77% of the time, an infinitely fast encoder makes the render at most $1/0.77 \approx 1.30 \times$ faster; speeding up composition has a ceiling of $1/0.23 \approx 4.35 \times$ (Figure 2). For photographs, where composition is 85–90% on the container, the encoder ceiling drops to

1.11–1.18 $\times$. There was also a concurrency argument: segments run one encoder per worker, and GeForce cards used to allow only three concurrent hardware encode sessions (five from 2023). NVIDIA’s current support matrix lists twelve, so that argument has weakened, but the Amdahl one had not. So VTV reports which hardware encoders FFmpeg can open on the machine, and keeps encoding on the CPU. Section 7.4 shows this has to be revisited once composition runs on the GPU.

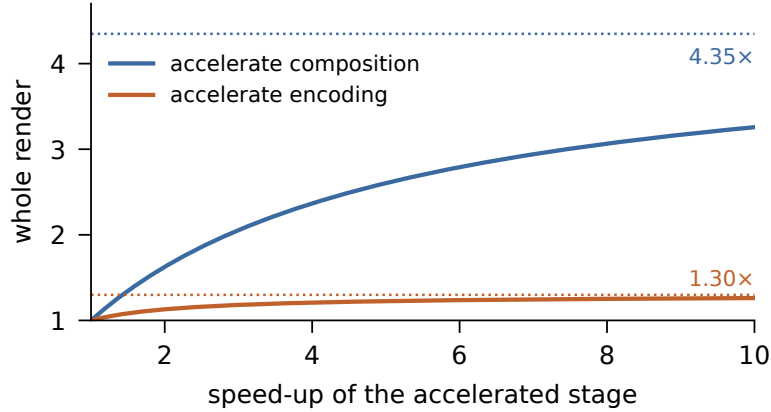


Figure 2: Whole-render speed-up from the measured 77/23 profile when only one stage is accelerated (analytical).

4 Rendering in segments

The rule I built around is: *a frame range that has been encoded is a file whose name says so*. A segment is finished if and only if `seg-NNNNN.mp4` exists, and that name is only ever created by an atomic rename from `.part.mp4` after the encoder exits with status zero. There is no separate resume routine to go stale. Every render asks of every segment whether it is already done, and a first attempt is just the case where the answer is always no.

Where segments start and end. Segments are cut only at clip starts, accumulating until a segment has at least $\text{round}(12 \cdot \text{fps})$ frames. Twelve seconds limits the work a crash can lose, gives about two hundred segments for a forty-minute video (enough to keep eight cores busy to the end), and keeps FFmpeg’s start-up of about 30 ms under half a percent of each segment. Boundaries are computed as frame indices, $\text{round}(\text{start} \cdot \text{fps})$, and each segment runs to the next one, so the segments cover $\text{range}(\text{total_frames})$ exactly. If each segment’s length were rounded from its own duration instead, a frame could be gained or lost per segment, and a two-hundred-segment video would drift seconds away from its narration.

When a checkpoint is stale. The scratch folder is named by a SHA-256 fingerprint of the timeline and settings with timestamps removed. A retry finds its own segments; any edit lands in a new folder. The fingerprint over-invalidates on purpose. Re-rendering something unnecessarily costs time. Reusing a stale segment ships the wrong picture, and the user cannot tell.

Parallel work and assembly. Because `describe(t)` is a pure function of the timeline and a time, frame n does not depend on frame $n - 1$, so segments are also the unit of parallel work. They run in a process pool of $\min(8, \text{cores}, \text{outstanding})$ workers, each encoding with `-c:v libx264 -preset veryfast -threads 1`. The final file is a concat-demuxer stream copy (`-c:v copy -movflags +faststart`), with the narration added in the same step, and a preview of the finished prefix is republished every 30 s. Tests compare decoded frames with `ffmpeg -f framemd5`: a resumed

render must equal an uninterrupted one, and a pooled render must equal an inline one. A separate harness kills the renderer mid-job and fails if any finished segment changes during the resume.

How well it scales. On my laptop (Intel Comet Lake, 8 logical processors) a 120-second typography video at 1080p, which is ten segments, took 96, 62, 54 and 52 s with 1, 2, 4 and 8 workers (Table 1). One worker was already faster than real time. The gains flatten. Ten equal segments cap the speed-up at 2, 3.3 and 5, and each worker keeps two processes busy (Python composition and an x264 encoder), so four workers already ask for all eight logical processors; that is probably most of the rest. I did not record per-core use. On the 2-vCPU container a 48-second render took 28.6 and 29.1 s with one worker and 27.5 s with two, as expected when one worker already fills both cores.

Workers	Wall time	Speed-up	Ceiling from 10 segments	Render time / video length
1	96.0 s	1.00×	1.0×	0.80
2	62.0 s	1.55×	2.0×	0.52
4	54.0 s	1.78×	3.3×	0.45
8	52.0 s	1.85×	5.0×	0.43

Table 1: Segmented render of a 120-second typography video at 1080p on the GTX 1650 laptop.

5 The GPU painter

5.1 Describe, then paint

The composer is split in two. `describe(t)` returns a *display list*: ordered layers (background, picture, drawn primitive, plate, label, vignette, message), a separate list of overlays (captions, credits, badges) that transitions never mix, the outgoing side of any transition, and the eased mix progress. It touches no pixels, fonts or files. A *painter* turns a display list into a frame. There are two painters, the Pillow reference and the GPU one, and both read the same display list (Figure 3). If a back-end ever needed its own description, the composer would have to know which backend it was describing for, and the split would be gone.

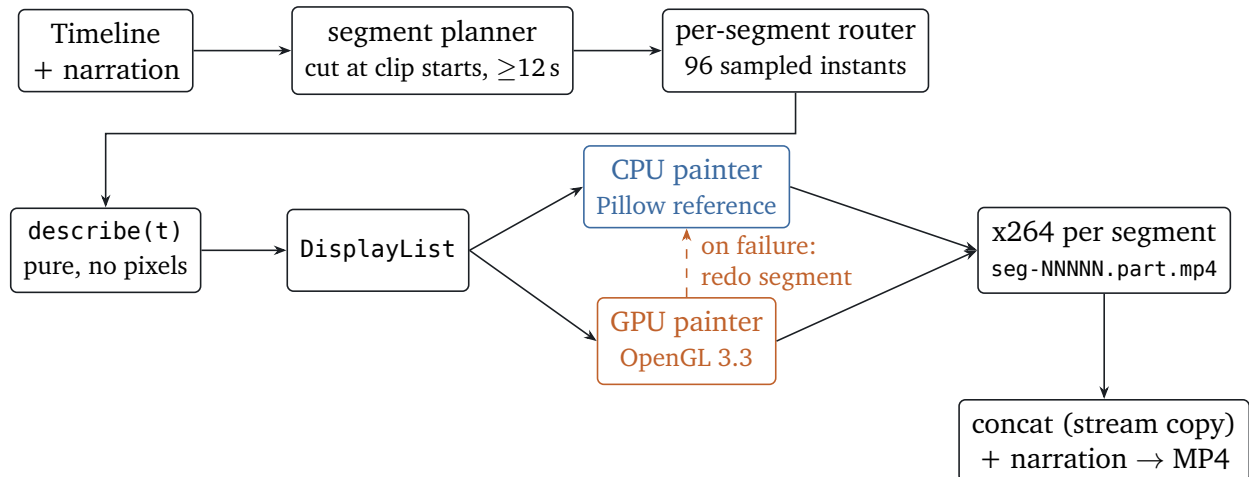


Figure 3: Render architecture. Both painters read the same display list. A segment goes to the CPU unless some sampled instant needs a photograph resampled, and a failed GPU segment is redrawn by the reference painter.

5.2 What runs on the card

I use OpenGL 3.3 through ModernGL, in a context with no window. OpenGL 3.3 is available on NVIDIA, AMD and Intel drivers from the last decade (I have only tested NVIDIA), ModernGL installs as one Python wheel, and nothing is tied to CUDA. The card does one job: **resampling photographs into the frame** (Table 2).

Work	CPU (Pillow)	GPU (OpenGL 3.3)
Layout, text, plates, labels	✓	
Animation engine (charts, diagrams, typography)	✓ (then uploaded)	
Overlays (captions, credits, badges)	✓ (after readback)	
Photograph resampling and placement (camera moves, crop)		✓
Vignette	✓	
Transition mix (blend, wipe, push)	✓ (each side composed first)	
Encoding (x264)	✓	

Table 2: Where each part of a frame is drawn.

Everything else stayed on the CPU for a reason. Text is 0.7% of composition, and drawing glyphs with the reference on both paths makes captions bit-identical. Overlays went back to the reference after the first hardware run: Pillow’s RGBA drawing *replaces* pixels and then flattens onto the theme background, while a GPU alpha-composites, and copying that quirk into a shader would mean reproducing a behaviour nobody chose. Transitions stayed because the reference wipe is a hard step with a blurred band, and doing it on the card meant three readbacks and two uploads to replace one `Image.blend`.

5.3 Matching Pillow’s resampler

The card’s built-in bilinear filter reads only the four texels nearest each output pixel, however much the image is being shrunk. On fine detail that aliases, and thin lines shimmer as a camera move drifts across them. On my test still, bilinear against Pillow’s Lanczos for a 0.8 downscale differed by up to 40 levels per channel, with a mean of 8. So the shader implements Lanczos-3,

$$L_3(x) = \frac{3 \sin(\pi x) \sin(\pi x/3)}{\pi^2 x^2} \quad (0 < |x| < 3), \quad L_3(0) = 1, \quad L_3(x) = 0 \text{ otherwise,}$$

and follows Pillow’s code in the details that change pixels. Output pixel i samples source coordinate $(i + 0.5)s$. When shrinking, the kernel is widened by $\max(1, s)$, or the result is undersampled whatever the kernel. Taps that fall outside the image are dropped and the remaining weights renormalised; an earlier version clamped them to the edge texel, which counted that texel twice and changed every border.

Two passes through an 8-bit buffer. Pillow resizes in two one-dimensional passes and writes an 8-bit image between them. Lanczos overshoots at sharp edges, and that intermediate clamps the overshoot before the second pass sees it. My first shader did the whole kernel in one floating-point pass. It matched Pillow on a gradient and on soft stripes, which never overshoot, and failed by up to 23 levels on a still with pure white lines. The shipped painter renders pass one into an RGBA8 framebuffer with blending off, clamps, and reads that buffer in pass two. The loop is capped at 64 taps, so a reduction beyond $10\times$ sends the whole frame to the CPU. The remaining difference is arithmetic: Pillow uses 22-bit fixed-point weights and integer sums, the shader uses float32.

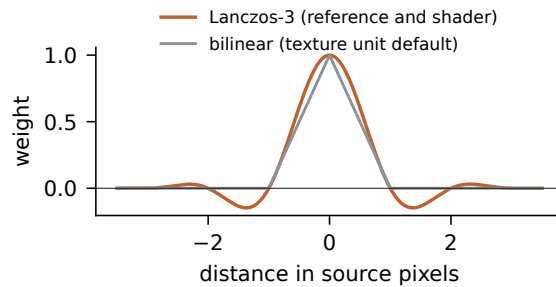


Figure 4: The Lanczos-3 kernel used by Pillow and the shader, against the bilinear tent a texture unit uses by default.

5.4 Routing, capability and fallback

A display list *needs resampling* only if a picture is present, including on the outgoing side of a transition, and the painter and the segment router use that one rule. The router samples 96 instants across a segment plus its last frame (one every eighth of a second over twelve seconds) and sends typography-only segments to the CPU.

A card being installed is not the same as a card that works. It can be driverless, too old for the shaders, inside a container with no render node, or out of memory. So at start-up `gpu_probe.probe()` creates a context, compiles the shaders, *draws a calibration frame and compares it with the CPU reference*. The GPU backend is registered only if that passes, and the answer is cached against the adapter and driver strings, so a driver update re-runs it. `VTV_DISABLE_GPU=1` turns it off. A GPU failure (lost context, out of memory, shader error) marks only that segment to be redrawn on the next backend, and every fallback is reported with its reason.

6 Deciding what “the same picture” means

A GPU will not be bit-identical to Pillow. The rule has two parts:

1. no channel of any pixel may differ from the reference by more than 2 (on 0–255);
2. **no pixel may exceed that:** the count of out-of-tolerance pixels must be zero.

The second part is the one that matters. A mean-difference test would pass a caption sitting one pixel high on every frame of a four-hour video. Big mistakes, like a missing overlay or a wipe running backwards, differ by hundreds and fail either part.

The calibration set has 23 scenes: background; message; four picture placements (fitted, zoomed, panned, contained); two plates; two text anchors; two vignette strengths; overlays over a picture and during a blend; and blend, wipe and push at 0, 50 and 100%. For each scene the harness reports the worst channel difference, the out-of-tolerance count, the mean absolute difference and a bounding box. A classifier then explains failures by transforming the GPU frame until the difference collapses: flips, ± 1 px shifts, a uniform offset (wrong clear colour), edges only, high frequencies only. If flipping takes the worst difference from 184 to 0, the cause is a mirrored readback. I tested both tools before trusting them: deliberately broken painters (a 2 px plate shift, dropped overlays, one that raises) must fail, and two reference painters must pass.

7 Results

7.1 First contact with the GTX 1650

The first run on my GeForce GTX 1650 (OpenGL 3.3, driver 610.88) passed 15 of 23 scenes. The pattern of failures gave the answer before I looked at a pixel: every failing scene had something

vertically asymmetric in it (a plate near the top, a line of text) and every passing scene was symmetric. A plate at rows 11–50 of a 180-row frame differed across rows 11–169, the plate and its mirror. Table 3 lists the four bugs.

#	Bug	Evidence	Fix
1	Framebuffer read bottom-up (OpenGL’s origin is lower-left, Pillow’s upper-left)	7 of 8 failures; bounding box exactly plate $\cup$ mirror	flip on readback
2	Cleared to black instead of the theme background (11, 13, 16)	18,432 of the 20,736 differing pixels in picture-contained	clear to the theme background
3	Picture drawn at its box, but the reference scales to cover and clamps the origin	the other 2,304 pixels, a 9-row band	<code>_fitted_box</code> reproduces the arithmetic
4	Overlays alpha-composited instead of replaced	overlay scenes off by 193	overlays drawn by the reference

Table 3: Bugs found on first contact with hardware. In picture-contained, $18,432 + 2,304 = 20,736$, the reported count exactly.

7.2 After the fixes

I re-ran the calibration set with a detailed test still of thin high-contrast lines, the worst case for resampling. On the GTX 1650 all 23 scenes passed: worst difference 2, zero pixels out of tolerance, mean absolute difference at most 0.07. I then ran the same painter and harness on Mesa’s llvmpipe (software OpenGL 4.5), a completely separate implementation of the API: all 23 passed, worst difference 1. The GPU benchmark later repeated the check on the GTX 1650 at 640×360 with its own photograph, and all 23 passed with a worst difference of 1. On every run, scenes without a photograph were bit-identical, and every non-zero difference was inside a resampled picture (Figure 5).

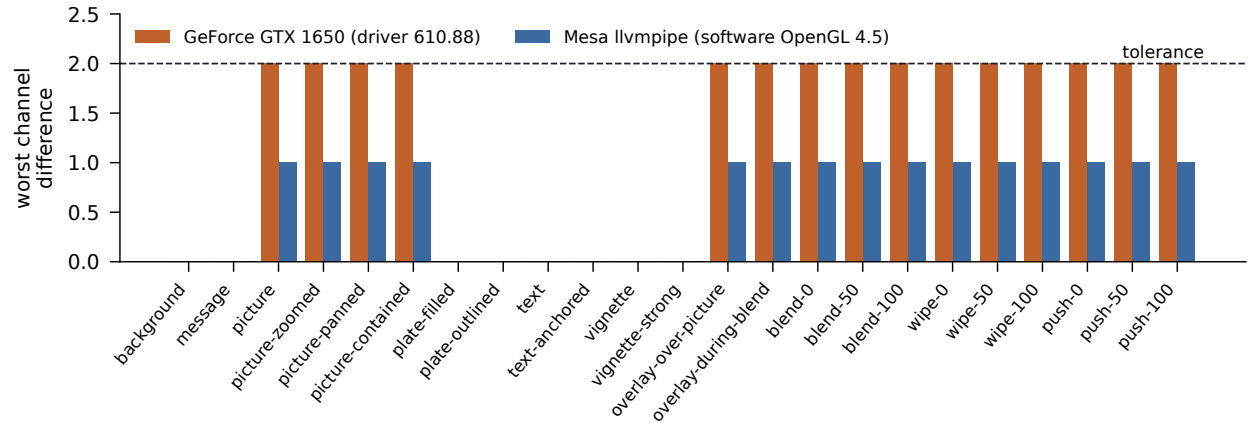


Figure 5: Worst per-channel difference from the CPU reference for all 23 scenes (320×180 , detailed still) on the GTX 1650 and on Mesa llvmpipe. Every scene has zero pixels outside tolerance on both.

The diagnosis script only saves images for scenes that fail, so the passing run left none. The images in the repository come from an earlier run that failed on the same still. Recomputed from those files, the worst differences are 11 to 23 levels across the 15 picture scenes, with 256 to 868 pixels out of tolerance per scene, mostly on the white lines where Lanczos overshoots (Figure 6). That matches the single-pass shader described above.

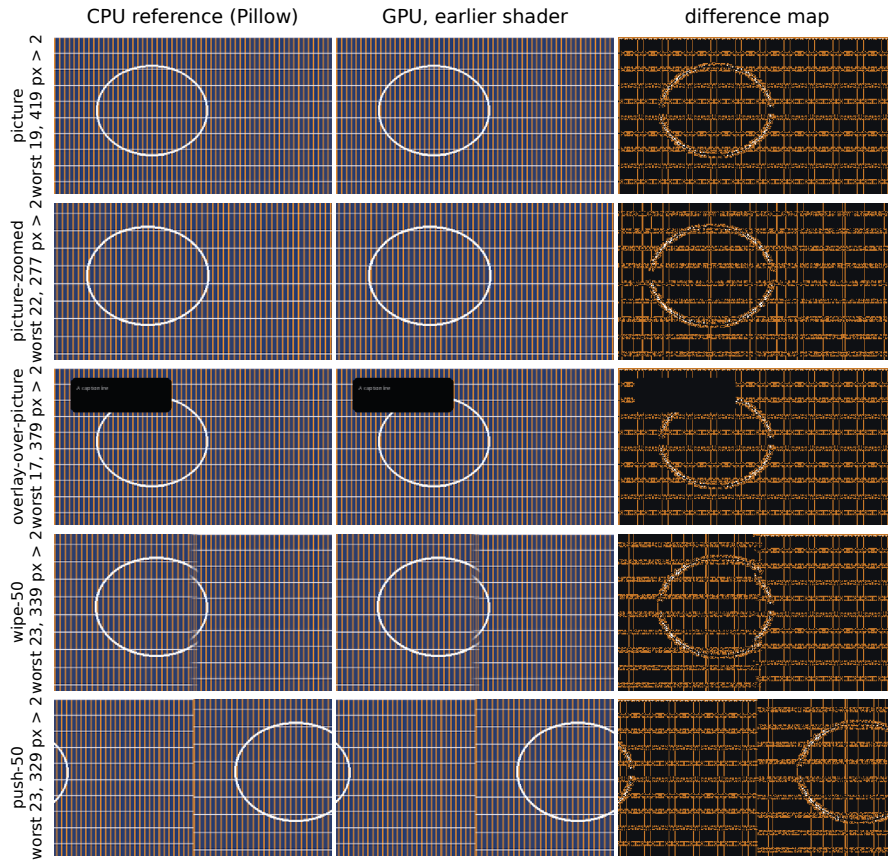


Figure 6: Frames from the earlier, failing run. Difference map: dark is identical, amber is 1–2 levels (within tolerance), white is more than 2. The failures sit on the high-contrast white lines.

7.3 Which details are necessary

To check that each matching detail earns its place, I built variants that change one thing and ran them on llvmpipe (Figure 7, Table 4). A floating-point intermediate that is still clamped *also passes*, so the clamp is what keeps the result in tolerance and the 8-bit quantisation only lowers the mean difference. Removing the clamp too fails 15 scenes by up to 23 levels, the same size as the earlier failing run. Clamping out-of-range taps to the edge fails 10 scenes by up to 57. The texture unit’s bilinear filter fails every picture scene by up to 84, with a mean difference of 6.9. The eight scenes that pass under every variant are exactly the ones without a photograph.

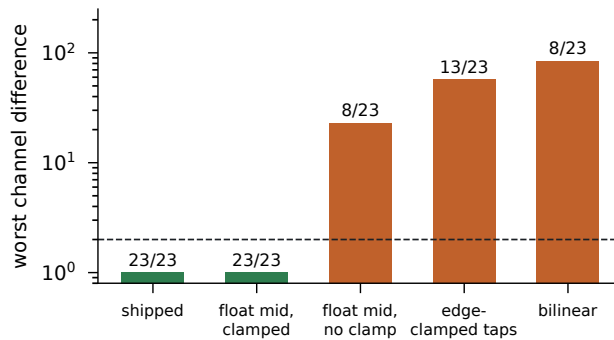


Figure 7: Worst channel difference (log scale) for each resampler variant on llvmpipe. Labels give scenes passing.

Variant	Passing	Worst	Out-of-tolerance px	Max mean diff.
Shipped (RGBA8 intermediate)	23	1	0	<0.001
Float intermediate, clamped	23	1	0	0.087
Float intermediate, no clamp	8	23	3,753	0.113
Taps clamped to the edge	13	57	15,211	0.497
Bilinear texture filter	8	84	677,297	6.883

Table 4: Resampler ablations on Mesa llvmpipe (23 scenes, detailed still).

7.4 How much faster the GPU is

I ran `scripts/paper_bench.py` on my laptop: GeForce GTX 1650 (4 GB, driver 610.88), Intel Comet Lake CPU with 8 logical processors, Windows build 26200. For each workload it times composition on the CPU painter and on the GPU painter, and x264 encoding of the same frames, at 720p and 1080p; each figure is the median of three repeats of 24 frames after a warm-up frame (Table 5, Figure 8).

	Workload	CPU (fps)	GPU (fps)	GPU / CPU	x264 (fps)
720p	typography	69.0	76.1	1.10×	115.7
	images	9.2	82.1	8.89×	56.1
	transitions	4.7	34.9	7.42×	44.5
	mixed	14.4	41.1	2.86×	45.8
1080p	typography	30.6	28.8	0.94×	48.4
	images	3.7	43.3	11.73×	24.2
	transitions	2.1	16.6	7.71×	26.8
	mixed	7.2	21.4	2.99×	22.6

Table 5: Composition on each painter and x264 encoding on the GTX 1650 laptop (medians of three repeats; GPU/CPU from unrounded values).

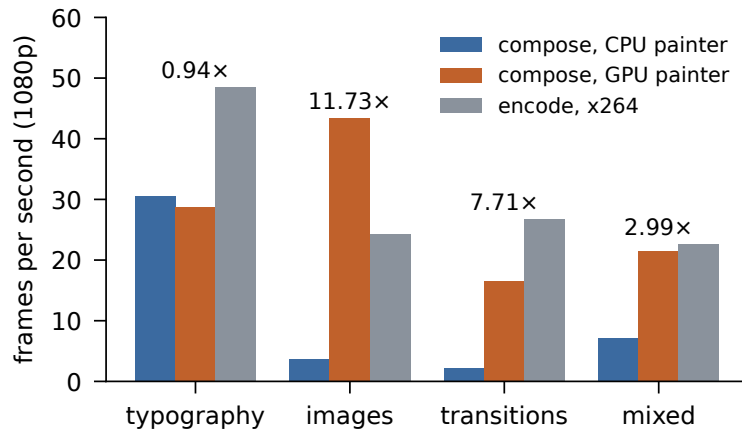


Figure 8: Composition on the CPU and GPU painters and x264 encoding at 1080p. Labels give GPU/CPU.

Photographs are where the card pays off: 11.7× at 1080p (10.0–11.9× across the three repeats), which agrees with the 11.6× I had noted in the code during development. The same note says typography was 2.2× slower; with the current painter I measured 0.94–1.10×. Transitions between photographs gain 7.4–7.7× and mixed content about 3×. Typography shows no reliable gain (1.10× at 720p, 0.94× at 1080p, noisy across repeats), because those frames are drawn by the animation engine on the CPU either way. That is why the router sends typography segments to the processor.

The bottleneck moves. With photographs on the GPU, composition stops being most of the work. At 1080p, if each frame is composed and then encoded, composition drops from 87% to 36% for images, from 93% to 62% for transitions and from 76% to 51% for mixed content, so x264 becomes the bigger half for photographs. The same Amdahl argument that ruled out a hardware encoder now points the other way: with photographs composed on the GPU, a free encoder could make an images-only render up to $2.79\times$ faster ($1.95\times$ for mixed content), against $1.15\times$ and $1.32\times$ before offloading on the same laptop. Adding compose and encode time per frame slightly overstates the total, because in the renderer they run in separate processes and overlap, but the direction holds. Timing NVENC against x264 on this card is my next experiment.

7.5 Long renders

Before this work the longest video VTV had rendered end to end was 150 seconds. The long-render harness renders on the desktop app with the GPU painter available, samples memory (process and children), scratch disk and VRAM every two seconds, and checks the output length with `ffprobe` at the end. Its test video alternates typography with a 1600×1000 test photograph under Ken Burns, pan and zoom moves, joined by 0.8 s dissolves, wipes and pushes, at 1080p30 and CRF 23. A 60-minute render drew all 296 segments in 71 minutes and then failed to upload, because a signed URL issued when the job was claimed had expired after an hour. URLs are now issued at upload time. The two- and four-hour runs after that fix are in Table 6 and Figure 9.

Video	Render	Segments	On GPU	Peak RSS	RSS slope	Peak VRAM	Length error
120 min	142.0 min	600	598	465 MB	-0.095 MB/seg	505 MB	0.00 s
240 min	341.4 min	1,200	1,198	471 MB	-0.113 MB/seg	916 MB	0.00 s

Table 6: Long renders on the GTX 1650 laptop. Peak scratch disk was 4.6 GB and 9.2 GB; the output files were 962 MB and 1,924 MB.

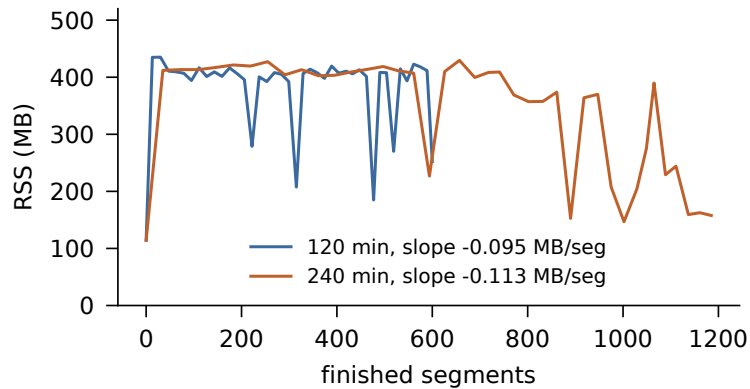


Figure 9: Resident memory against finished segments during the long renders.

Memory does not grow with the number of segments, and the output is exactly as long as the timeline. Almost every segment went to the GPU, which fits the routing rule: every typography shot in the test video opens with a transition from a photograph. Scratch disk grew linearly with length, which sets a real disk requirement for multi-hour videos.

7.6 Real projects

Figure 10 puts the long renders next to five renders of real projects stored in the development database (1080p30, standard quality, with real photographs, generated images, captions and credits). Those took $3.9\text{--}8.9\times$ their own length; a forty-minute project (2,399 s of video) took 10,116 s. This is not a controlled comparison. The real projects used large photographs and denser layouts,

the painter and worker count were not recorded, and they predate the GPU verification. The figure shows the range of costs the system has actually paid, not a speed-up.

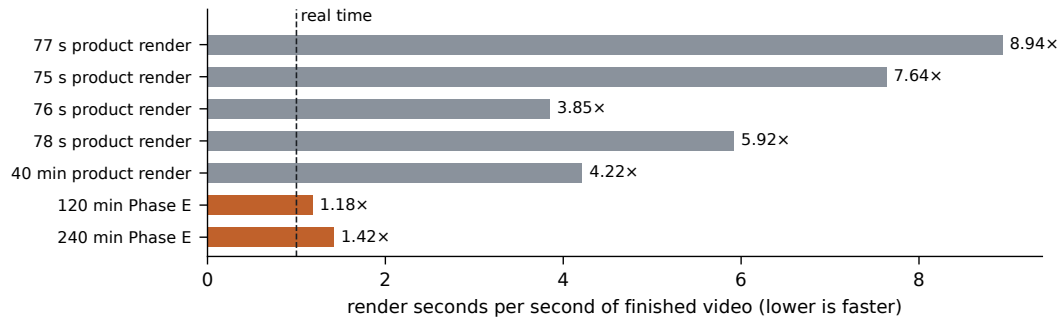


Figure 10: Render time per second of finished video. Grey: real projects (backend not recorded). Orange: long-render tests with the GPU painter. Different content and settings.

8 What I would tell someone doing the same

1. **Profile per content kind before buying hardware.** The popular fix, a hardware encoder, addressed the smaller share. Removing wasted buffer work gave 2.2x for free.
2. **Parallelise across processes by cutting on frame indices.** One decomposition gave me resumable renders, parallelism and a unit of fallback, and exact frame boundaries keep the audio in sync by construction.
3. **Move only the expensive, parallel part to the GPU,** and keep the reference for everything that must match exactly.
4. **Make correctness a contract with zero exceptions,** and only use a device after it passes a calibration frame at run time. That found four real bugs no test on a machine without a GPU could have found.
5. **Re-profile after every speed-up.** The GPU made photographs 11.7x faster to compose, and the encoder became the next thing to fix.

9 Limits

- **One machine for speed.** The throughput and laptop scaling numbers come from one GTX 1650 laptop, three repeats each, with no control of CPU frequency or temperature. Typography timings were noisy (GPU/CPU from 0.79x to 1.35x across repeats). Other GPUs will give other ratios.
- **Few devices for correctness.** The GTX 1650 and llvmpipe (software, not a GPU). AMD, Intel and macOS (OpenGL capped at 4.1) are untested.
- **Small test frames.** Equivalence was measured at 320x180 and 640x360. The long renders ran at 1080p but checked length and memory, not pixels.
- **Synthetic content.** The benchmarks and long renders use test content. The 77/23 project profile and the 12→26 frames/s result are recorded in the code and docs, but the raw profiler output was not kept.
- **No NVENC measurement yet,** which the new profile says is now worth doing.

10 Next

Time NVENC against x264 per segment on the GTX 1650, now that encoding is the larger share for photographs. Measure GPU contexts against worker count. Move the animation-engine primitives onto the card without changing the display list. Try wgpu (D3D12, Vulkan, Metal) as the portable successor to OpenGL.

AI use

I used AI coding and writing assistants while building Voice-to-Video and drafting this report. I checked every number against the repository, and all measurements come from the project's own scripts.

A Where every number comes from

Claim	Source in the repository
77% / 23%; 12→26 fps; $1.3\times$ ceiling	src/vtv/adapters/media/compute.py; docs/GPU_RENDERING.md
Text 0.7% of composition 10 ms of a 42 ms frame; vignette 19 ms, 43 s	docs/GPU_RENDERING.md; gpuPainter.py docstring src/vtv/animation/painter.py; src/vtv/animation/canvas.py
12 s segments; ~30 ms FFmpeg start; 40-min video at $4\times$ its length	src/vtv/adapters/render/segments.py
Eight-worker cap	src/vtv/contracts/render.py
Per-content profile (container)	paper/experiments/e1-cpu-profile.json
Laptop CPU/GPU/x264 fps, 640×360 check, scaling	paper/experiments/e5-gtx1650-paper-bench.json
Container scaling	paper/experiments/e2-render-scaling.txt
Bilinear up to 40 (mean 8); single pass off by 19–23; 64 taps; $10\times$ limit	src/vtv/adapters/render/gpuPainter.py
Tolerance 2, zero outliers, 23 scenes	src/vtv/animation/equivalence.py; gpuProbe.py
First run 15/23; four bugs; 18,432 + 2,304	docs/GPU_RENDERING.md; gpuPainter.py docstring
23/23 on GTX 1650 after the fixes	paper/gtx1650-diagnosis/report.json
Earlier failing run (Figure 6)	PNGs in paper/gtx1650-diagnosis/, recomputed
23/23 on llvmpipe; ablations	paper/experiments/llvmpipe-diagnosis/report.json; e4-ablation-llvmpipe.json
60 min in 71 min, 296 segments, URL expiry	docs/VERIFY_PHASE_D.md
Two- and four-hour renders	paper/long-render/phase-e-report.*
Real project render times	development database (renderJob records), not kept in the repository
NVENC session limits	Tom's Hardware (24 March 2023); NVIDIA Video Encode and Decode GPU Support Matrix (accessed 25 September 2026)

B Re-running it

```
pip install moderngl psutil
python -c "from vtv.adapters.render.gpuProbe import describe; print(describe())"
python -m unittest tests.testPainterEquivalence -v
python scripts/diagnose_gpuPainter.py --photo
python paper/experiments/cpuProfile.py --out e1.json
python paper/experiments/ablation.py e4.json
python scripts/paperBench.py
python scripts/verifyLongRender.py --minutes 120,240
```